RelayNet Demo Procedure

0. Before the faculty arrives

Open **Git Bash** and go to the project:

```
cd /d/RelayNet
```

Check that the repository is clean:

```
git status
```

You want:

```
nothing to commit, working tree clean
```

Then launch the dashboard:

```
python -m streamlit run dashboard/app.py
```

Keep this terminal open.

Streamlit will show something similar to:

```
Local URL: http://localhost:8501
```

Open that URL in Chrome.

1. Start with the project explanation — 30 seconds

Before clicking around, say:

"Our project is RelayNet, an Adaptive Emergency Mesh Routing Protocol prototype for infrastructure-less disaster communication. We simulate a source, destination and three UAV relay nodes. The system observes RSSI, battery, queue length, mobility, link stability

and hop count. These observations are processed through a temporal Knowledge Graph and independent multi-agent Q-learning to select an appropriate relay."

Then show this mentally:

```
Dynamic MANET
      ↓
Network observations
      ↓
Temporal Knowledge Graph
      ↓
Contextual reasoning
      ↓
Multi-agent Q-learning
      ↓
Relay selection
      ↓
Packet forwarding
      ↓
Performance evaluation
```

Say:

"We then evaluate the system under five disaster scenarios and independently validate conventional MANET protocols using NS-3."

2. Dashboard — Overview

Start on the **Overview** tab.

You will see the six KPIs:

- PDR
- Packet loss
- Average delay
- Throughput
- Energy

- Lifetime

What to say

"This is the main monitoring view. The sidebar allows us to change the disaster scenario and routing method. These KPIs summarize the corresponding simulation results."

Then change the scenario

Select:

High Mobility

Keep:

KG + MARL

This is a good scenario to demonstrate because KG+MARL performs particularly well here.

Say:

"For example, under high mobility, the network becomes more difficult because relay positions and wireless links change more frequently."

3. MANET tab — show the network

Click:

MANET

Show the topology image.

Say:

"This is the dynamic MANET layer. The source communicates with the destination through UAV relays. Wireless links are determined dynamically using range-based neighbour discovery."

Point to the route.

Explain:

"The system can discover multi-hop routes and respond to relay availability and failures."

Then scroll down slightly and show:

Routing log

Say:

"The routing log records the active nodes, available links and selected route at different simulation time steps."

Important

Don't spend 2 minutes explaining the graph.

Just establish:

dynamic topology → route discovery → forwarding.

4. Knowledge Graph tab

Click:

Knowledge Graph

Show the graph.

Say:

"The next layer is the temporal Knowledge Graph. Network observations are stored as temporal subject-predicate-object triples."

Give one concrete example:

"For example, a relay can have a particular RSSI, battery level, queue length and link stability. The reasoner derives contextual facts such as stable link, low queue or sufficient battery."

Then show the predicate selector.

Select something like:

`HAS_BATTERY`

or:

`HAS_RSSI`

Show the table.

Say this

"This is useful because the system doesn't only produce a final numerical decision. The intermediate contextual information is also inspectable."

That's your **explainability** point.

5. Q-Learning tab

Click:

Q-Learning

Show the parameters.

Point out:

```
Learning rate = 0.15
Discount factor = 0.90
Epsilon start = 0.20
```

Don't read every parameter.

Say:

"Each UAV relay maintains an independent Q-table. The agents learn from forwarding outcomes."

Then show the training graph.

Say:

"This visualization shows the training metrics including reward, delivery rate, TD error and exploration."

Then show the Q-table.

Say:

"The verified training run learned 44 Q-table entries."

If they ask how KG and RL are combined

Give the equation:

```
FinalScore = 0.65 × KGScore + 0.35 × QScore
```

Then say:

"The KG contributes contextual knowledge while the Q-learning component contributes learned experience."

6. Performance tab

Click:

Performance

Select:

Packet Delivery Ratio

Show the graph.

Then select:

Throughput

Show that graph.

Say:

"We did not evaluate the system using one random run. We performed a multi-seed evaluation."

Then say the important number:

"Five scenarios, four methods and ten seeds give us 200 Python experiment runs."

You can mention the four methods:

```
Static
Weighted Baseline
Contextual KG
KG + MARL
```

7. Demonstrate the most important result

Go back to **Overview**.

Select:

```
Scenario: High Mobility
Method: KG + MARL
```

Say:

"High mobility is particularly interesting because the network topology changes rapidly."

Then give the actual result:

"KG+MARL achieves a PDR of approximately 20.48% in this scenario, slightly higher than the other evaluated methods."

Don't say "best everywhere."

Immediately add:

"However, it does not win every scenario. Under low-battery conditions, the current reward shaping needs improvement."

This makes your presentation scientifically credible.

8. NS-3 Validation — important

Now click:

NS-3 Validation

This is your newest dashboard feature and should definitely be demonstrated.

At the top, explain:

"We also performed an independent NS-3.47 validation."

Point out:

AODV
OLSR
DSR

Then explain:

"This is deliberately separate from the Python AEMRP implementation. We are not claiming that the KG and Q-learning algorithm is running natively inside NS-3."

This sentence is **very important**.

9. Show NS-3 scenario comparison

Select:

Normal

Show the protocol table.

Then select:

High Mobility

Show it again.

Say:

"We evaluated five scenarios, three protocols and ten runs per combination, resulting in 150 NS-3 runs."

Then give the key result:

"AODV produced the highest average PDR in all five NS-3 scenarios, while high mobility was the most difficult scenario."

10. Show protocol ranking

Scroll to:

Protocol ranking

Change the metric if necessary.

For example:

`Packet Delivery Ratio`

Show the ranking.

Say:

"The dashboard also exposes the statistical protocol ranking generated from the NS-3 analysis."

This demonstrates that you aren't just displaying a single graph—you have actual analysis.

11. Show the seven NS-3 graphs

Scroll down.

You'll see:

1. Packet Delivery Ratio
2. Packet Loss Ratio
3. Average Delay
4. Average Jitter
5. Throughput
6. Energy Consumed
7. Network Lifetime

You don't need to explain all seven individually.

Say:

"The NS-3 analyzer generates seven performance views covering reliability, latency, throughput, energy and network lifetime."

12. Explain the Python vs NS-3 difference

If they notice:

"Why are the Python PDR values around 20–28% while NS-3 values are around 4–10%?"

Answer immediately:

"They should not be directly compared numerically. The two simulators use different topology, traffic, radio, mobility and packet models. NS-3 is being used as an independent MANET baseline validation layer."

This is the correct answer.

13. Finish with the architecture

Return to **Overview**.

Give your final 20-second summary:

"So the complete RelayNet pipeline is dynamic MANET observation, temporal Knowledge Graph reasoning, independent multi-agent Q-learning, adaptive relay selection and performance evaluation. We evaluated 200 Python configurations across multiple seeds and independently validated MANET behavior with 150 NS-3 runs."

Then:

"The current prototype demonstrates adaptive and competitive relay selection, particularly under high mobility, while low-battery reward shaping remains an identified improvement area."

Stop there.

Your ideal 5–7 minute demo flow

Time	Screen	What you demonstrate
0:00–0:30	Introduction	Problem + objective
0:30–1:10	Overview	KPIs + scenario
1:10–1:50	MANET	Dynamic topology + routing
1:50–2:30	Knowledge Graph	Context + explainability
2:30–3:10	Q-Learning	Training + Q-table
3:10–3:50	Performance	200-run evaluation
3:50–4:50	NS-3	AODV/OLSR/DSR + 150 runs
4:50–5:30	NS-3 graphs	Seven metrics
5:30–6:00	Conclusion	Contribution + limitation

What NOT to do during the demo

❌ Don't run this

```
python -m python.experiments.multi_seed_evaluation
```

during the presentation.

You already have the verified 200-run results.

❌ Don't run all NS-3 experiments live

They are already completed.

❌ Don't open source code first

Show the **working system first**. Open code only if they specifically ask.

❌ Don't claim

"KG+MARL is always the best."

It isn't.

❌ Don't claim

"We implemented KG+RL inside NS-3."

You didn't.

❌ Don't claim

"This is a real UAV emergency communication system."

It's a simulation/research prototype.

If they suddenly ask for a live experiment

You have the **Simulation Lab** tab.

You can say:

"The dashboard also provides controlled execution of individual verified Python experiments."

Then choose a relatively lightweight experiment such as:

MANET topology

and click:

Run experiment

But only do this if they specifically request a live execution. **The pre-generated results are safer and faster for the normal demo.**

Keep these numbers on a small cheat sheet

Memorize these:

```
3 UAV relays
5 disaster scenarios

Python:
4 methods
10 seeds
200 runs

NS-3:
3 protocols
10 runs
```

```
150 runs

KG:
330 raw triples
216 derived triples
546 total

RL:
44 Q-table entries

Final score:
65% KG
35% Q-learning

High mobility:
KG+MARL PDR = 20.48% ± 1.44%
```

And the most important sentence:

"KG+MARL is adaptive and competitive, particularly under high mobility, but the current reward function needs further optimization for low-battery conditions."

If you follow this exact order, your demo will feel like a **complete system demonstration**, rather than a collection of separate experiments.

now can u tell me all te features which are there in project clearly
Worked for 9s

Absolutely. Here is the **complete feature map of your RelayNet project**, separated into what is actually implemented, what is evaluated, and what is only future work.

RelayNet — Complete Feature List

1. 🚨 Disaster MANET Simulation

This is the foundation of the project.

The system simulates an **infrastructure-less disaster network** where normal communication infrastructure may be unavailable.

Network components

You have:

- **1 source**
- **1 destination**
- **3 UAV relay nodes**
    - R1
    - R2
    - R3

The UAVs act as mobile relay nodes between the source and destination.

Implemented MANET features

Your simulator includes:

- Dynamic neighbour discovery
- Wireless communication range calculation
- 3-D node positions
- Dynamic wireless links
- Link-cost calculation
- Multi-hop routing
- Dijkstra-based route discovery
- Route selection
- Relay availability
- Relay failure handling
- Node mobility
- Link-stability changes
- Packet generation
- Packet forwarding
- Queue behaviour
- Probabilistic packet delivery
- Energy consumption tracking
- Network lifetime tracking
- Routing-log generation
- MANET topology visualization

So the basic flow is:

```
Source
  ↓
Discover available UAV relays
  ↓
Evaluate wireless links
  ↓
Calculate route
  ↓
Select relay
  ↓
Forward packet
  ↓
Destination
```

2. 📡 Network-State Monitoring

RelayNet does not make relay decisions using only distance.

It monitors multiple network conditions.

Each relay has information such as:

RSSI

Represents wireless signal strength.

Battery

Represents remaining relay energy.

Queue length

Represents congestion/backlog at the relay.

Mobility

Represents how much the relay is moving.

Link stability

Represents how reliable the wireless connection is.

Hop count

Represents routing distance.

These parameters become the input to your intelligent relay-selection system.

3. 🧠 Multiple Relay-Selection Methods

Your project doesn't only implement KG+MARL.

It compares **four different methods**.

1. Static

A fixed/static relay-selection strategy.

2. Weighted Baseline

Uses manually weighted network conditions.

The current weighting is:

```
RSSI              25%
Battery           20%
Queue             15%
Mobility          10%
Link stability    20%
Hop count         10%
```

3. Contextual KG

Uses Knowledge-Graph-derived contextual information to select relays.

4. KG + MARL

Combines:

```
Knowledge Graph
       +
Multi-Agent Q-Learning
```

This is the main intelligent approach in the project.

4. ☸ Temporal Knowledge Graph

This is one of your major features.

The KG represents the network state as:

```
Subject → Predicate → Object
```

For example conceptually:

```
R1 → HAS_BATTERY → 82
R1 → HAS_RSSI → strong
R1 → HAS_QUEUE_LENGTH → low
R1 → HAS_MOBILITY → low
```

The graph is **temporal**, meaning network observations can change over simulation time.

Your project tracks states at different simulation points such as:

```
T0
T5
T10
```

5. 🔎 KG Contextual Reasoning

The Knowledge Graph doesn't just store raw measurements.

It derives higher-level contextual information.

Examples include:

- SUFFICIENT_BATTERY
- LOW_QUEUE
- STABLE_LINK
- LOW_MOBILITY
- STRONG_RSSI

This allows the system to reason about relay suitability.

For example:

```
Strong RSSI
+
Low queue
+
Stable link
+
Sufficient battery
        ↓
Good relay candidate
```

6. 🎯 Relay Recommendation

The KG reasoning produces relay recommendations such as:

```
PREFER
CAUTION
AVOID
```

This makes the routing decision more interpretable.

Instead of only saying:

"R2 has score 0.82"

the system can reason:

"R2 has sufficient battery, low queue and stable connectivity, therefore it is preferred."

7. 🤖 Multi-Agent Q-Learning

You have **independent Q-learning agents**.

Each UAV relay maintains its own Q-table.

So conceptually:

```
Agent R1 → Q-table
Agent R2 → Q-table
Agent R3 → Q-table
```

The agents learn from network outcomes.

Important RL parameters

Your verified configuration includes:

```
Learning rate       = 0.15
Discount factor     = 0.90
Epsilon start       = 0.20
Epsilon decay       = 0.97
Minimum epsilon     = 0.02
Training episodes   = 60
Packets / episode   = 250
```

The training uses:

```
Random seed = 42
```

The verified training run learned **44 Q-table entries**.

8. ⚖️ KG + Q-Learning Decision Fusion

This is the central intelligence mechanism.

The system combines the KG decision with learned experience:

```
FinalScore
    =
0.65 × KGScore
+
0.35 × QScore
```

Therefore:

```
Network observations
        ↓
       KG
        ↓
 Contextual score
        ↓
         ┐
         ├──→ Final relay score
         ┘
   Q-learning
        ↓
    Q-score
```

The KG provides contextual knowledge while Q-learning provides learned experience.

9. 🌪️ Disaster Scenarios

Your project evaluates five different network conditions.

Normal

Normal network operation.

High Mobility

Relay nodes move more aggressively and links change more frequently.

High Congestion

Network queues/congestion become more challenging.

Low Battery

Relay energy becomes a major constraint.

Relay Failure

One or more relay paths become unavailable.

These scenarios allow you to test whether the routing strategy adapts to different network conditions.

10. 📊 Multi-Seed Statistical Evaluation

You don't rely on a single simulation.

Your Python evaluation contains:

```
5 scenarios
×
4 methods
×
10 seeds
=
200 runs
```

The four methods are:

```
Static
Weighted Baseline
Contextual KG
KG + MARL
```

Metrics measured

You evaluate:

- Packet Delivery Ratio
- Packet-loss ratio
- Average delay
- Throughput
- Energy consumption
- Network lifetime

This is important because it makes the evaluation more statistically meaningful than showing one simulation run.

11. 📈 Performance Visualization

The project generates visualizations for:

- PDR
- Packet loss
- Average delay
- Throughput
- Energy consumption
- Network lifetime
- Relay scores
- Routing comparisons

These are available through the dashboard and stored in `results/`.

12. 🛰️ Independent NS-3 Validation

You added a separate NS-3 validation layer.

This is **not** the same implementation as the Python AEMRP system.

NS-3 evaluates conventional MANET protocols:

```
AODV
OLSR
DSR
```

Evaluation size

```
5 scenarios
×
3 protocols
×
10 runs
=
150 NS-3 runs
```

So overall your project has:

```
Python evaluation = 200 runs
NS-3 evaluation   = 150 runs

Total              = 350 evaluated runs
```

13. 📡 NS-3 Metrics

The NS-3 analyzer produces seven major metrics:

1. Packet Delivery Ratio

How successfully packets reach the destination.

2. Packet Loss Ratio

How many packets are lost.

3. Average End-to-End Delay

Packet delivery latency.

4. Average Jitter

Variation in packet delay.

5. Throughput

Successful data delivery rate.

6. Energy Consumed

Energy usage during the simulation.

7. Network Lifetime

How long the network remains operational.

14. 📊 NS-3 Statistical Analysis

Your NS-3 analysis generates:

`ns3_multiseed_summary.csv`

and:

`ns3_protocol_rankings.csv`

So you don't just run NS-3 and look at raw output.

You perform:

```
Raw NS-3 runs
      ↓
Multi-seed aggregation
      ↓
Mean + standard deviation
      ↓
Protocol ranking
      ↓
Graphs
```

15. 🖼 Seven NS-3 Graphs

The NS-3 analysis generates seven graphs:

```
ns3_pdr_percent.png
ns3_loss_ratio_percent.png
ns3_average_delay_ms.png
ns3_average_jitter_ms.png
ns3_throughput_kbps.png
ns3_energy_consumed_j.png
ns3_network_lifetime_s.png
```

These are displayed directly inside your dashboard.

16. 🖥 Streamlit Control Center

You now have a complete interactive dashboard.

The dashboard contains **seven sections**.

1. Overview

Shows:

- Scenario selection
- Routing method selection
- PDR
- Packet loss
- Delay
- Throughput
- Energy
- Lifetime
- Method comparison
- Decision pipeline

2. MANET

Shows:

- Dynamic topology
- Available wireless links
- Routing path
- Routing log

3. Knowledge Graph

Shows:

- KG visualization
- KG relations
- Predicate filtering
- Triple data

4. Q-Learning

Shows:

- RL hyperparameters

- Training progress
- Reward
- Delivery rate
- TD error
- Exploration
- Q-table

5. Performance

Shows:

- Six Python evaluation metrics
- Multi-seed graphs
- Summary CSV download

6. NS-3 Validation

Shows:

- Scenario selection
- AODV / OLSR / DSR comparison
- Best PDR
- Best throughput
- Best lifetime
- Protocol ranking
- Seven NS-3 graphs
- NS-3 summary CSV download

7. Simulation Lab

Allows you to execute selected Python experiment modules from the dashboard.

For example:

```
MANET topology
Knowledge Graph
KG visualization
KG + MARL training
Multi-seed evaluation
```

17. 🔄 Experiment Execution From Dashboard

The Simulation Lab provides controlled experiment execution.

Instead of manually typing every module command, you can select an experiment and run it.

The dashboard captures:

- stdout
- stderr
- return code
- timeout errors

and displays the result.

There is also a **180-second execution limit** for dashboard-launched experiments.

18. 🗂 Automatic Result Generation

The project stores experiment outputs under:

`results/`

including:

- CSV datasets
- routing logs
- KG data
- Q-tables
- RL metrics
- evaluation summaries

- topology images
- performance graphs
- NS-3 outputs
- NS-3 analysis

So the project has a clear:

```
Simulation
     ↓
Result generation
     ↓
Analysis
     ↓
Visualization
     ↓
Dashboard
```

pipeline.

19. 🔬 Reproducibility

Your experiments use explicit seeds.

Python:

```
RL seed = 42
10 seeds for multi-seed evaluation
```

NS-3:

```
10 runs per protocol/scenario
```

This makes the experiments repeatable rather than relying entirely on uncontrolled randomness.

20. 🛡️ Failure and Edge-Case Handling

The project includes handling for:

Relay failure

Unavailable relays can be excluded from routing.

Low battery

Battery affects relay selection and network lifetime.

Congestion

Queue state influences relay suitability.

Mobility

Mobility and link stability influence decisions.

NS-3 low-battery behavior

The NS-3 implementation was adjusted to safely handle node depletion rather than forcing an invalid Wi-Fi PHY state.

NS-3 DSR accounting

Application-level packet accounting was used to avoid incorrect transmission accounting observed with the initial FlowMonitor-only approach.

21. 📋 What is NOT a feature

This is just as important for your viva.

These are **not currently implemented**:

Item	Status
AEMRP natively inside NS-3	❌
KG directly connected to NS-3	❌
Q-learning directly connected to NS-3	❌
Neo4j	❌
MADDPG/deep MARL	❌
Physical UAV deployment	❌
Real UAV communication	❌
Wi-Fi Direct deployment	❌
Bluetooth Mesh deployment	❌
Production emergency network	❌

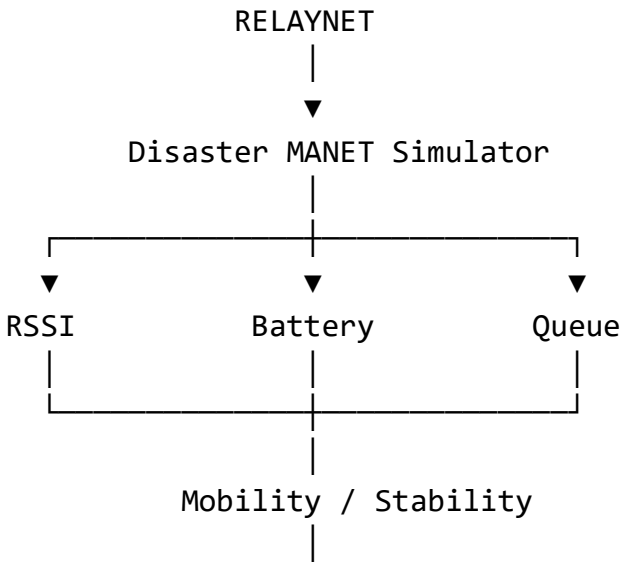
These are **future work**, not current features.

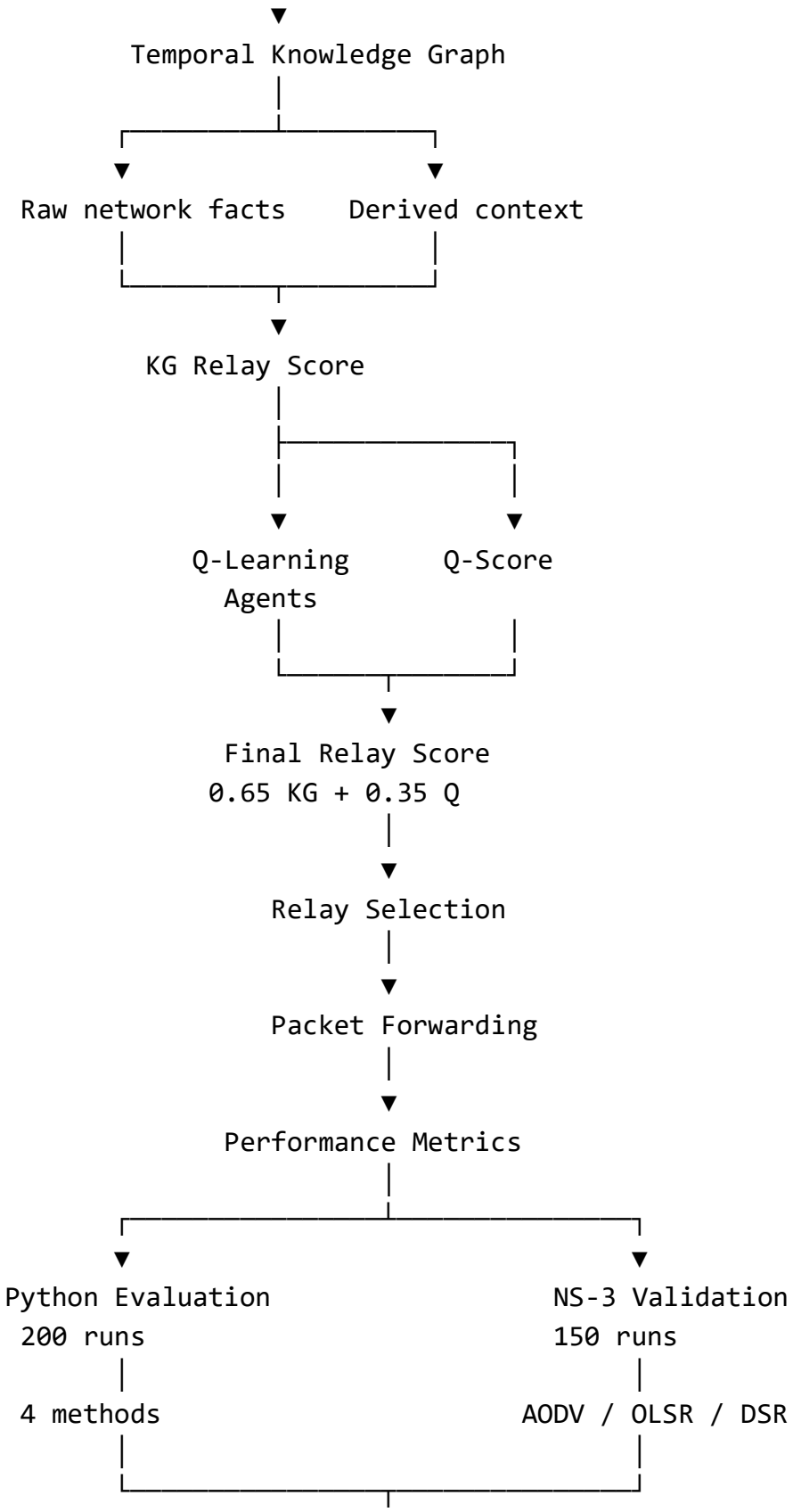
The whole project in one picture

If the examiner asks:

"Explain your complete system."

Use this:




Temporal Knowledge Graph
Raw network facts
Derived context
KG Relay Score
Q-Learning Agents
Q-Score
Final Relay Score
0.65 KG + 0.35 Q
Relay Selection
Packet Forwarding
Performance Metrics
Python Evaluation
200 runs
4 methods
NS-3 Validation
150 runs
AODV / OLSR / DSR

▼

Streamlit Dashboard

The simplest way to remember your project

Your project has **five major layers**:

1. Network layer

Dynamic disaster MANET with UAV relays.

2. Intelligence/context layer

Temporal Knowledge Graph.

3. Learning layer

Independent Q-learning agents.

4. Evaluation layer

200 Python runs + 150 NS-3 runs.

5. Presentation/control layer

Streamlit dashboard with seven interactive sections.

If you remember those five layers, you can reconstruct almost the entire project explanation during your viva.